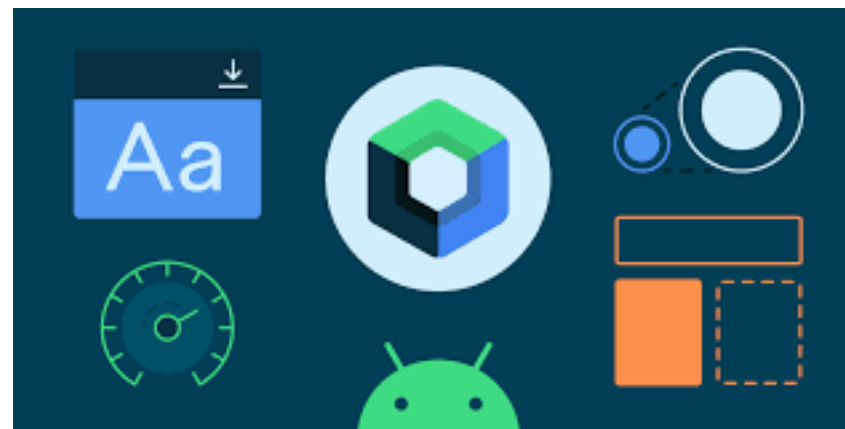




INTRODUCCIÓN A JETPACK COMPOSE

*UTN°2 Desarrollo de aplicaciones Android
Programación Multimedia y dispositivos móviles (PGL)*

INTRODUCCIÓN



- ¿Qué es JetPack Compose?
- Es un conjunto de herramientas moderno para crear una interfaz de usuario nativa de Android. Jetpack Compose simplifica y acelera el desarrollo de la interfaz de usuario en Android con menos código, herramientas potentes y API de Kotlin intuitivas.
- Es programación totalmente **declarativa** , lo que significa que puede describir su interfaz de usuario invocando un conjunto de elementos componibles. Durante los últimos diez años hemos estado utilizando una forma tradicional de diseño de interfaz de usuario imperativo.

PROGRAMACIÓN IMPERATIVA VS PROGRAMACIÓN DECLARATIVA

- IU imperativa
- Este es el paradigma más común. Implica tener un prototipo/modelo independiente de la interfaz de usuario de la aplicación. Este diseño se centra en el cómo más que en el qué. Un buen ejemplo son los diseños XML en Android. Diseñamos los widgets y componentes que luego se representan para que el usuario los vea e interactúe.

Xml:

```
<TextView
    android:id="@+id/tv_name"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
/>
```

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val tvName = findViewById<TextView>(R.id.tv_name)
        tvName.text = "Hola Mundo"
    }
}
```

PROGRAMACIÓN IMPERATIVA VS PROGRAMACIÓN DECLARATIVA

- UI declarativa
- Este patrón es una tendencia emergente que permite a los desarrolladores diseñar la interfaz de usuario en función de los datos recibidos. Este, por otro lado, se centra en el qué. Este paradigma de diseño utiliza un lenguaje de programación para crear una aplicación completa.
- En JetPack Compose no es necesario ningún código xml para crear vistas. Podemos crear vistas usando composable.

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            Text(text = "Hello World")  
        }  
    }  
}
```

VENTAJAS DE JETPACK COMPOSE

- Es muy rápido y ofrece un rendimiento fluido.
- Es sencillo de aprender.
- Es posible interoperar con un enfoque imperativo.
- Ofrece una mejor manera de implementar principios de acoplamiento flexible.
- Está 100% hecho en Kotlin, lo que lo convierte en un enfoque moderno en el desarrollo de Android.



FUNCIÓN COMPOSABLE

Syntax:

```
@Composable
fun MethodName(parameter: String) {
    //your content
}
```

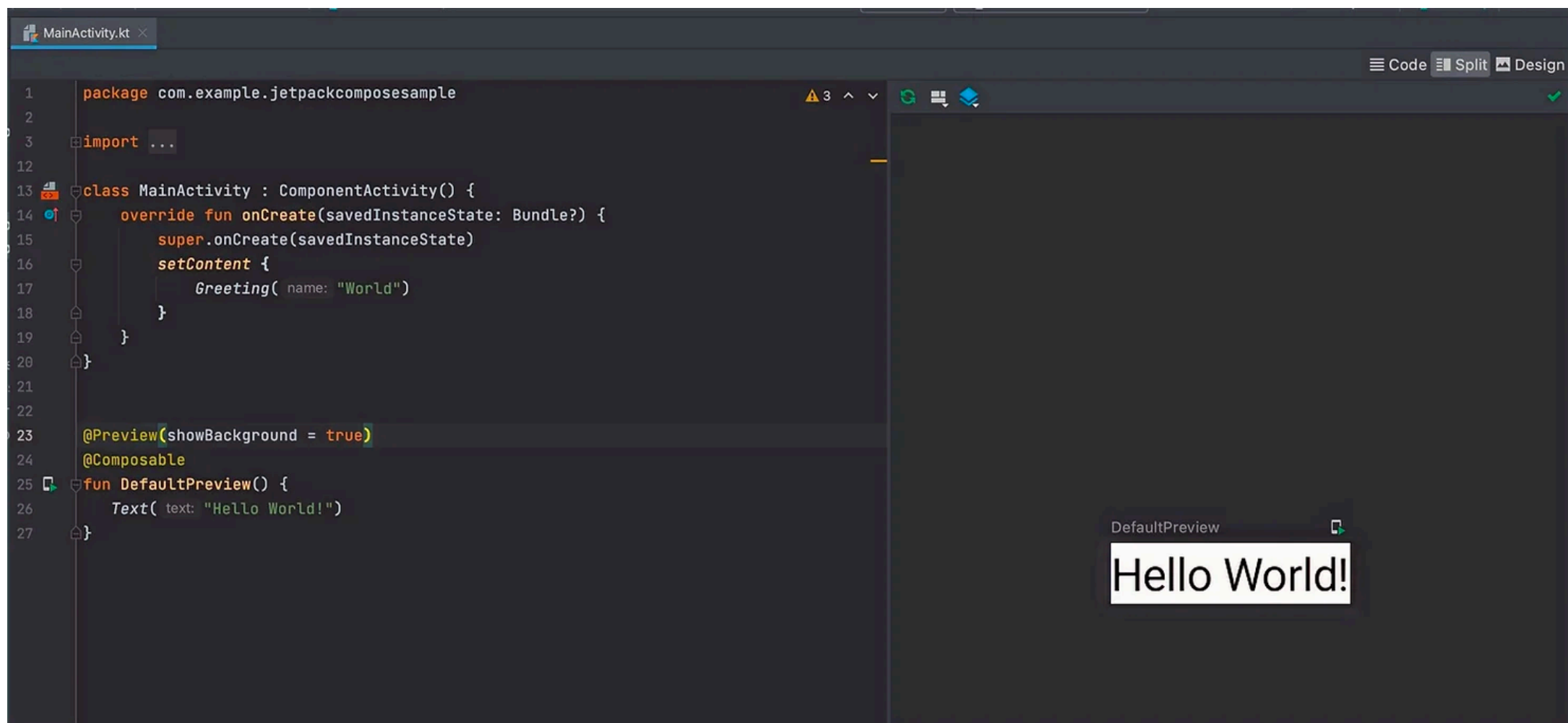
Example:

```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello $name!")
}
```

- Es lo mismo que cualquier función en programación. Pero necesitamos anotar con la anotación **@Composable**.
- En el ejemplo de la imagen creamos una nueva función componible: `Greeting()`.
- Usamos `Text()` como contenido. Es una función componible incorporada. Podemos agregar funciones componibles a otra función componible.

VISTA PREVIA EN JETPACK COMPOSE

- En Jetpack Compose podemos ver la vista previa de nuestro código en Android Studio. Nos permite ver el resultado sin ejecutar nuestra aplicación.
- Haciendo click en Split para ver el código y la vista previa.



¿CÓMO AGREGAR UNA VISTA PREVIA?

- Debe agregar la anotación `@Preview()` antes de la función componible. Después de agregar esta anotación, podremos ver la vista previa de nuestra interfaz de usuario.
- Las funciones `@Preview()` hasta ahora no pueden visualizar funciones que reciben parámetros. Pero es tan sencillo como crear una función por encima con los parámetros definidos como podemos ver en el siguiente ejemplo.

```
@Preview()
@Composable
fun GreetingPreview() {
    Compose1Theme {
        Greeting(name: "Android")
    }
}
```


¿CÓMO PERSONALIZAR? (I)

- Hay dos formas de personalizar la vista previa.
- Configure la personalización escribiendo manualmente.
 - ☑ **Nombre:** si proporcionas el argumento de nombre, mostrará el nombre de pila ("Preview1") en su área de vista previa. Podemos agregar varias vistas previas en la misma clase, por lo que si proporciona el nombre podrá identificarlo fácilmente.
 - ☑ **Fondo:** La función componible no define ningún fondo por sí sola, pero puedes agregar uno a la vista previa configurando `showBackground = true` en la anotación Vista previa. También hay un argumento `backgroundColor` para cambiar el color.

```
@Preview(name = "Preview1")
```

```
@Preview(name = "Preview1", showBackground = true,  
background-color = 0xFF03A9F4)
```

¿CÓMO PERSONALIZAR? (II)

```
@Preview(name = "Preview1", showBackground = true, widthDp = 200, backgroundColor = 0xFF03A9F4, heightDp = 50)
@Composable
fun DefaultPreview() {
    Text(text = "Hello world")
}
```

```
@Preview(name = "Preview1", device = Devices.PIXEL, showSystemUi = true)
@Composable
fun DefaultPreview() {
    Text(text = "Hello world")
}
```

☑ **Alto y Ancho:** La función componible establece su ancho y alto de forma predeterminada en un `wrap_content`. Si desea cambiar, pase los valores deseados `widthDp` y `heightDp`.

☑ **Interfaz de usuario del sistema y dispositivo:** Puede obtener una vista previa de una función componible en una pantalla ficticia (con una barra de estado, una barra de herramientas y un menú de navegación). Simplemente configure `showSystemUi = true` y estará listo. También puede cambiar el marco del dispositivo utilizado, por ejemplo, `dispositivo = Dispositivos.PIXEL`.

¿CÓMO PERSONALIZAR? (III)

- ☑ Múltiples vistas previas: Puede agregar varias vistas previas. Simplemente agregue la anotación `@Preview` en su elemento componible.

```
@Preview(name = "Preview1", showBackground = true,
backgroundColor = 0xFF2196F3)
@Composable
fun Preview1() {
    Text(text = "Hello world 1")
}

@Preview(name = "Preview2", showBackground = true,
backgroundColor = 0xFF2196F3)
@Composable
fun Preview2() {
    Text(text = "Hello world 2")
}
```